

MAJOR PROJECT SUBMISSION · 2026

Ferrite Browser: Capability-Governed Agentic Browsing with Verifiable Audit Trails

A Developer-Focused Rust Browser Built on Servo with Security-First Architecture

Built in Rust. Governed by policy. Verified by cryptography.

Ferrite is the first browser designed from the ground up to treat AI agents as first-class principals — with formal capability-based access control and tamper-evident audit logs.

Rust-Native

Memory-safe, high-performance engine

Capability-Governed

Every agent action token-gated

Verifiable

Cryptographic audit trails

Browsers Were Not Built for AI Agents

The browser was designed 30 years ago for humans clicking links. In 2026, AI agents autonomously browse, fill forms, make purchases, and execute complex multi-step workflows — all inside that same architecture. The result is a governance vacuum.

The Core Security Failure

Agentic browsers like **Comet**, **Atlas**, and **Dia** grant AI agents the same privileges as the human user — with no formal governance model, no audit trail, and no capability-based access control. Traditional browser security (same-origin policy, sandboxing) becomes irrelevant when an AI agent executes with full user privileges.

- 📄 OpenAI admitted in Dec 2025 that prompt injection "**may never be fully solved**" at the model level — making architectural enforcement essential.

Real Attacks in 2025

- **CometJacking** — exfiltrated emails & calendar data via hidden instructions (LayerX, Oct 2025)
- **Screenshot Injection** — hidden text hijacked Comet & Fellou agents (Brave, Oct 2025)
- **Tainted Memories** — poisoned Atlas's long-term memory via CSRF (LayerX, Oct 2025)
- **HashJack** — instructions hidden in URL fragments (Cato Networks, Nov 2025)

The Agentic Browser Landscape (2025–2026)

Every major browser player is racing to add AI agents. None have solved the fundamental security problem. Five competitors analyzed:

Perplexity Comet

Chromium-based. Launched Jul 2025. Executes shopping, booking, email tasks. **Vulnerable:** CometJacking, Amazon lawsuit for unauthorized bot purchases. No capability model.

ChatGPT Atlas

OpenAI's browser agent mode. Uses RL-based red teaming + adversarial model updates. **Admits:** prompt injection "unlikely to ever be fully solved." No cryptographic audit trail.

Dia (Atlassian)

Replaced Arc browser (\$610M acquisition, Sep 2025). AI-first with memory & skills. Integrates Jira/Linear. Lighter on security research. No formal capability disclosures.

Brave + Leo AI

100M+ users. Privacy-first with Shields. Publishes competitor vuln disclosures. Developing "fine-grained permissions" for agentic browsing — **not yet shipped**.

Chrome / Edge

Gemini + Copilot integrations. Google published agentic security architecture paper (Dec 2025). Massive market share but **incremental** approach to agent governance.

📌 **Key insight:** Every major player is adding AI agents to browsers. **None** have implemented formal capability-based access control or verifiable audit trails for agent actions.

Research Gaps Ferrite Fills

Academic research has advanced browser security and prompt injection defenses in parallel — but no prior work bridges them into a unified architectural solution for agentic browsing.

Browser Compartmentalization

Capsicum (USENIX 2010), **COWL** (OSDI 2014), **RLBox** (USENIX 2020) established capability sandboxing for OS and web workers. **Gap:** None extended to agentic contexts where AI acts with user privileges.

Browser Agent Benchmarks

WASP, **BrowseSafe**, **SecureWebArena** (2025) document attack surfaces comprehensively. **Gap:** These benchmarks describe the problem but propose no architectural defense mechanism.

Prompt Injection Research

Greshake et al. 2023 — first systematic indirect injection study. **Nasr et al. Oct 2025** (OpenAI/Anthropic/DeepMind) — tested 12 defenses; all fail against adaptive attacks. **Gap:** All defenses are model-level. No architectural enforcement exists.

Transparency & Auditability

Certificate Transparency (RFC 6962) and **Sigstore/Rekor** established append-only Merkle logs for certificate and supply chain verification. **Gap:** Never applied to browser agent action auditability.

📄 **Core research gap:** No system provides capability-mediated governance of browser agent actions with cryptographic verifiability. Ferrite fills this gap by combining the Capsicum/COWL lineage with CT-style audit logs.

Ferrite Browser: How It Works

Ferrite enforces a single architectural principle: **the Capability Broker is the sole privileged boundary**. All untrusted code — extensions, AI agents, web content — accesses every sensitive operation only through explicitly granted, unforgeable capability tokens.



Capability Token System	Wasm-Sandboxed Extensions	Verifiable Audit Log
Unforgeable tokens carry principal ID, capability type, origin scope, rate limits, URL allowlists, and expiry. Every privileged action requires a valid token. No ambient authority exists.	Extensions run as WebAssembly modules via Wasmtime/WASI. Capabilities injected through host functions only. 7+ capability types: dom.read, network.fetch, storage.write, and more.	Hash-chained append-only log (upgrading to Merkle tree) records every grant, denial, and exercise. Cryptographic proofs enable post-hoc verification that every agent action was authorized.

Six Security Gaps Ferrite Solves

A systematic comparison of what Ferrite provides versus the current state of every major browser and agentic product on the market today:

Security Gap	Existing Browsers	Ferrite's Solution
Capability-based agent security	MV3 permission model (Chrome/Brave); no formal system (Comet/Atlas)	Broker-minted scoped tokens + WASI sandbox. Agents are first-class principals.
Verifiable agent audit trail	NetLog is diagnostic, not cryptographic. Comet/Atlas have no tamper-evident logs.	CT-style hash-chained → Merkle log. Cryptographic proof of every agent action.
Prompt injection hardening	Model-level training + classifiers — OpenAI admits insufficient.	Risk-tiered gating, step-up consent in trusted UI isolated from web content.
Extension Wasm sandboxing	Process isolation + permission prompts (MV3). Coarse-grained.	Per-capability, per-origin, rate-limited Wasm component grants. Finer-grained.
Content blocking as primitive	Brave Shields blocks ads but not connected to a formal audit system.	adblock-rust engine with every block logged to the cryptographic audit system.
Governed AX tree for agents	No existing browser provides governed accessibility tree access for agents.	AX tree exposed to agents only through capability-gated broker. First-of-kind.

Build vs. Reuse: A Principled Stack

What We Build (Core Contribution)

1 Capability Broker + Token System

The central enforcement point — written entirely in Rust, zero dependencies on external privilege systems.

2 Risk Classifier + Agent Governance

Rego/OPA policy bundles via Regorus. Low/medium/high risk tiers with step-up consent UI.

3 Verifiable Audit Log

Hash-chain evolving to Merkle tree with rs_merkle. Cryptographic proof generation and verification.

4 DevTools + Agent Protocol

Capability Inspector, Agent Timeline, Audit Log Viewer built in Iced. JSON-RPC over WebSocket.

What We Reuse (Open Source Foundation)

Browser Engine

Servo v0.0.5 (MPL-2.0) — Track A primary. **CEF/Chromium** (BSD) — Track B validation harness proving engine-agnostic architecture.

Sandbox & Policy

Extism/Wasmtime (Apache-2.0) for extension isolation. **Regorus** (MIT) as pure-Rust OPA interpreter.

Network & Crypto

Tokio, hyper, rustls, quinn — async, HTTP, TLS, QUIC. **rs_merkle** for proof trees. **tracing + OpenTelemetry** for telemetry.

Storage & UI

rusqlite for audit index. **keyring** for credentials. **adblock-rust** (MPL-2.0) for content blocking. **Iced** (MIT) for UI framework.

8 Months · 3 Engineers · 24 Person-Months

A three-person team — R1 (systems/engine), R2 (security/crypto), R3 (UI/research) — working on Linux-first with Docker devcontainer for fully reproducible builds.



Expected Outcomes and Evaluation Targets

Three Core Deliverables

- 1

Working Browser Prototype

Capability-brokered agent governance on both Servo (Track A) and CEF (Track B), proving engine-agnostic architecture.
- 2

Two Standalone Developer Tools

Extension Security Analyzer (CLI) and Agent Action Inspector (UI)
— usable independently of the browser.
- 3

Research Paper

First formal model for capability-based governance of LLM browsing agents. Targeting **NDSS 2027** or **ACSAC 2026**.

Quantitative Targets

100%

Enforcement Coverage

No privileged action executes without a valid capability token

<1ms

Broker Latency

Median capability check overhead target

20+

Attack Scenarios

Adversarial test cases including indirect prompt injection

≥10%

Bandwidth Savings

Reduction on tracker-heavy sites via adblock-rust

CONCLUSION

Architectural Enforcement Is the Missing Layer

When OpenAI says prompt injection "may never be fully solved," they're talking about model-level defenses. Ferrite's capability-based constraints provide a complementary architectural layer — limiting the blast radius of any successful injection regardless of what malicious instructions say.

What Ferrite Proves



Architecture Over Training

Formal capability governance enforces security independently of model behavior — a defense-in-depth layer that works even when models fail.



Engine-Agnostic Design

The Servo + CEF dual-track strategy proves the capability broker architecture is portable across rendering engines and deployable broadly.

Future Work Beyond 8 Months

- Formal verification of capability token properties
- Public transparency log integration (Sigstore Rekor)
- Privacy-by-default partitioned state (per-origin storage isolation)
- Energy-aware scheduling with per-tab resource budgets
- Windows and Android platform support



Open source under **dual MIT/Apache-2.0** license. Community contributions welcome from day one.